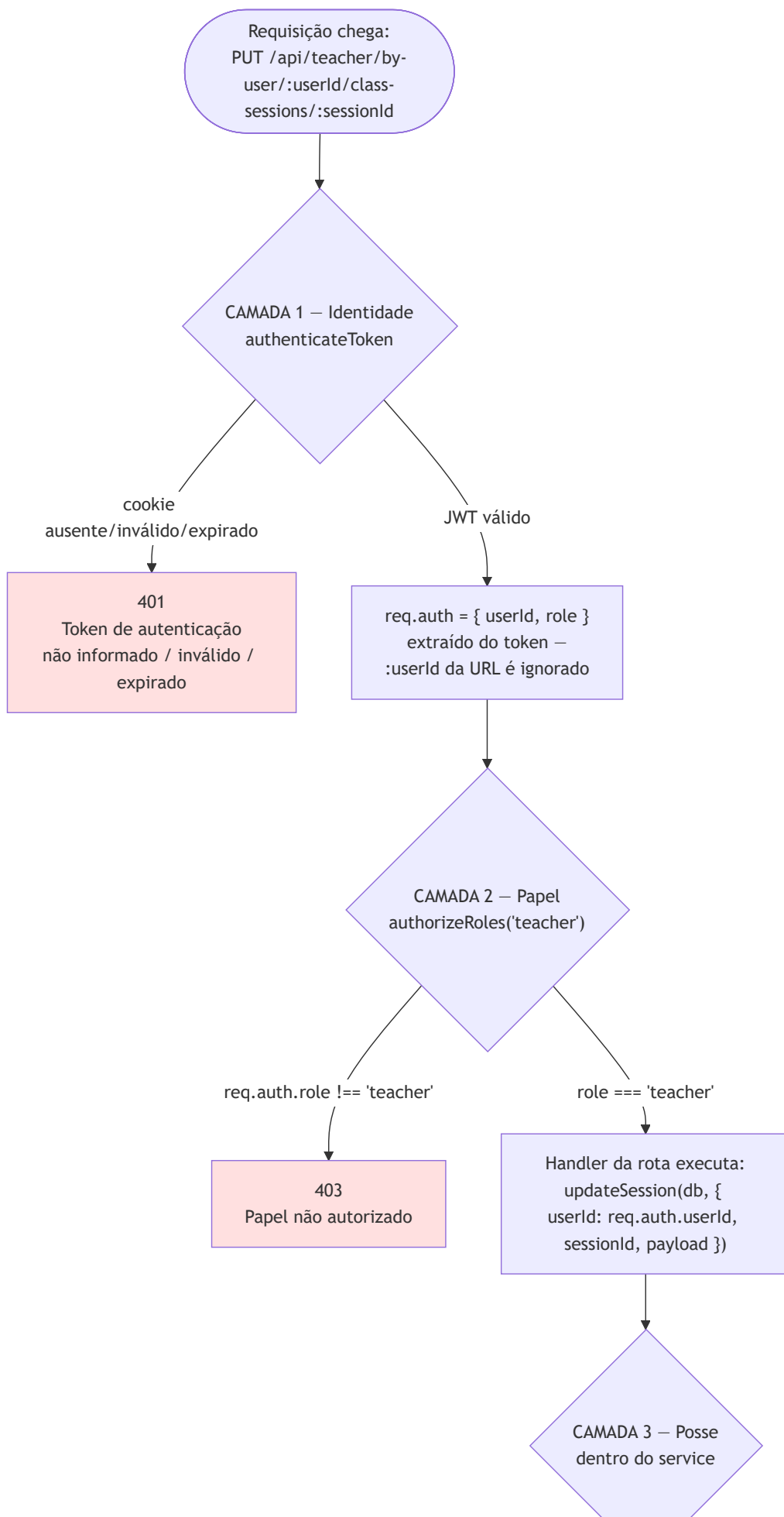
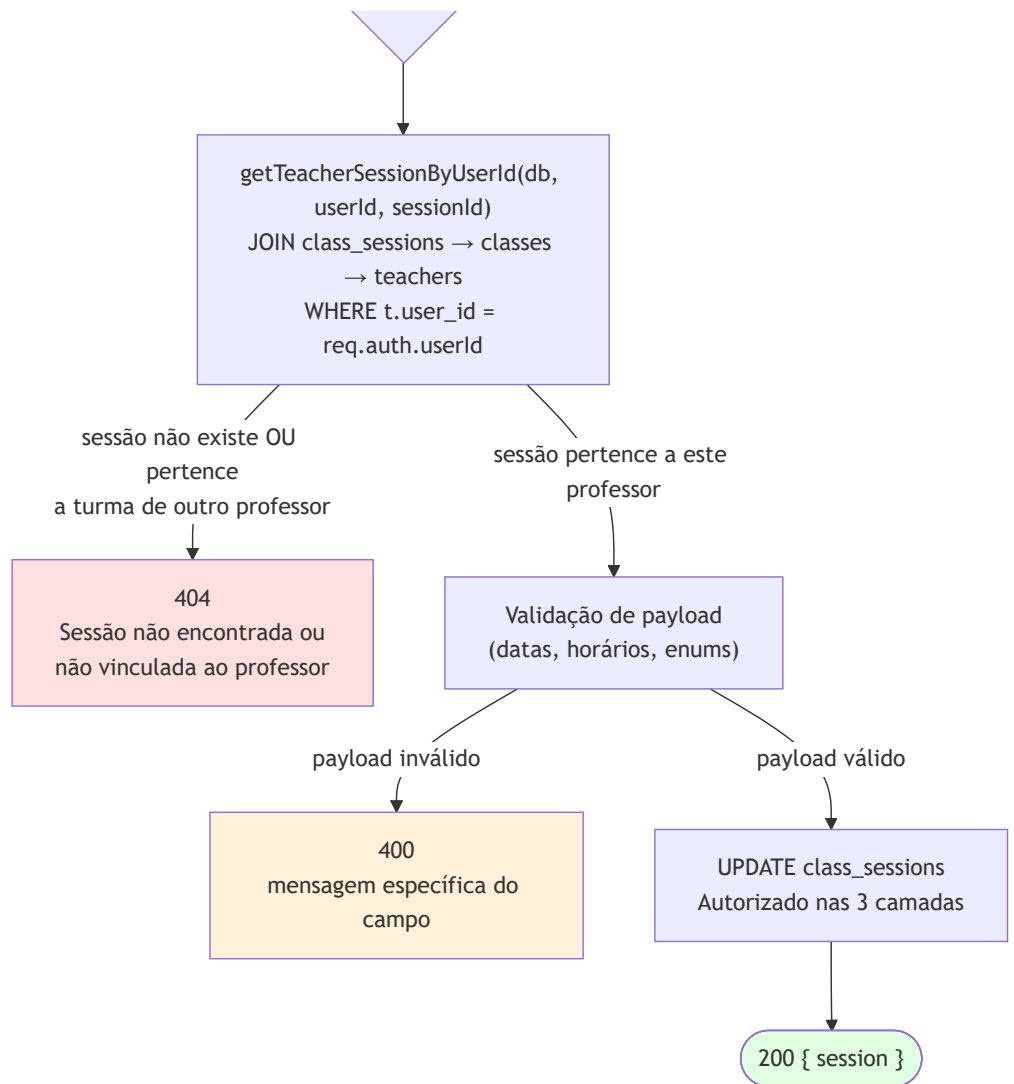


Diagrama — Fluxo de Autorização

Autorização no CourseHub acontece em três camadas distintas, nessa ordem, e as três precisam passar para qualquer operação sensível ser permitida. Este diagrama usa um exemplo concreto real (professor editando uma sessão de turma) para tornar as três camadas inequívocas.





Que problema este diagrama esclarece

"Autorização" no CourseHub não é uma checagem única — é fácil, ao ler só o middleware `authorizeRoles`, presumir que qualquer professor autenticado pode editar qualquer sessão de qualquer turma. Este diagrama existe para deixar claro que **isso é falso**: existe uma terceira camada, dentro de cada service, que confirma posse individual do recurso, e ela é tão obrigatória quanto as outras duas. Sem esse terceiro passo, um professor autenticado e com o papel certo ainda não teria permissão de fato.

Como interpretar os elementos

- As três camadas são coloridas de forma consistente com o resultado: vermelho claro para negações de identidade/papel/posse (401/403/404), laranja para erro de validação (400, uma categoria diferente — não é sobre "quem pode", é sobre "o dado está correto"), verde para sucesso.

- **req.auth.userId é a única fonte de identidade em toda a árvore** — o diagrama marca explicitamente " :userId da URL é ignorado" porque esse é o ponto de segurança mais importante do sistema: mesmo que alguém troque o :userId na URL manualmente, a consulta de posse na Camada 3 usa req.auth.userId , não o valor da URL.
- A Camada 3 usa 404, não 403, quando a sessão pertence a outro professor. Essa é uma escolha real do código (getTeacherSessionById retorna null tanto para "não existe" quanto para "não é seu"), não uma imprecisão do diagrama — evita vaziar a informação de que o recurso existe, mas pertence a outra pessoa.

Que decisões arquiteturais este diagrama evidencia

- **Nenhuma camada confia na anterior além do que ela garante:** a Camada 2 confirma o papel, mas não sabe nada sobre posse de recursos — essa responsabilidade é exclusivamente da Camada 3, e vive dentro do service, não do middleware. Middlewares no CourseHub nunca fazem consulta de posse a um recurso específico.
- **A checagem de posse é reimplementada por domínio** (getClassAccessService.getClassOwnedByTeacher para turmas, getStudentIdByUserId + verificação de matrícula para alunos, etc.) em vez de um middleware genérico de "dono do recurso" — decisão consistente com o resto da arquitetura (cada domínio é responsável pela sua própria regra de negócio, incluindo quem pode tocar no quê).
- **Admin não passa pela Camada 3:** rotas /api/admin/* param na Camada 2 (authorizeRoles("admin")) — não existe conceito de "posse" para um admin, ele opera sobre qualquer recurso do sistema por definição do papel.

Trade-offs que este modelo representa

- **Checagem de posse duplicada em espírito através dos domínios** (a mesma forma de "resolver dono, comparar, negar se não bate" se repete em classAccessService , activityScopeService , courseContentScopeService) — mais previsível de auditar domínio por domínio, mas significa que uma mudança na filosofia de autorização (por exemplo, permitir professores substitutos) precisaria tocar múltiplos arquivos, não um só.
- **Nenhum cache de resultado de posse dentro de uma mesma requisição:** se um handler precisasse confirmar posse duas vezes (não é o caso comum, mas acontece em transações com múltiplos passos), a consulta de posse roda de novo — simplicidade de código em troca de uma pequena redundância de consulta em casos raros.
- **404 em vez de 403 para "existe mas não é seu":** melhor para não vaziar existência de recursos alheios, mas significa que os logs de erro não distinguem automaticamente "tentativa de acesso

indevido" de "erro de digitação no ID" — ambos aparecem como 404 idênticos.

Como este diagrama deve evoluir

Se um quarto papel for reintroduzido no futuro (o texto de `business-rules.md` nota que `manager / staff` foram removidos por falta de uso, não por decisão de nunca mais existirem), este diagrama precisa de uma nova Camada 2 mostrando onde esse papel se encaixa na árvore de decisão — e, mais importante, se ele passa ou não pela Camada 3 de posse (dependeria inteiramente do que esse papel deveria poder fazer). Se uma funcionalidade de "professor substituto com acesso temporário a uma turma" for implementada, a Camada 3 deste diagrama precisa ganhar um novo ramo além de "é o professor titular" — hoje esse caso não existe no código.